Групов проект - Ecolator

Изготвен от следните ученици на 11А и 11В клас випуск 2027 **Технологично училище „Електронни системи“**
към Технически университет – София (ТУЕС):

Виктор Сираков (А, № 6)
Кристиян Кобарелов (А, № 18)
Борис Маринов (В, № 7)
Виктор Алексиев (В, № 9)

Научни ръководители (Преподаватели):
Александра Александрова
Даниел Димитров
Георги Станачков
Слави Димитров
Калоян Ангелов
Милен Спасов
Марио Бенов

Учебно заведение: Технологично Училище “Електронни системи”
към Технически Университет - София (ТУЕС)
Предмет: Единен проект за 11-ти клас
(ООП, БД, РС, ВОТ)

Дата: 19.04.2026

Съдържание:

Корица....1
Заглавна страница....1
Съдържание....2
Описание на проекта....3
1. Проучване....3
   1.1. Предметна област и Целева аудитория....3
   1.2. Преглед на съществуващи решения....3
   1.3. Tech Stack....4
2. Проектиране....5
   2.1. Функционални изисквания към системата / програмата....5
   2.2. Архитектура на системата....6
   2.3. Инфраструктурна диаграма....6
   2.4. Схема на БД....7
   2.5. UML Class диаграма....8
3. Реализация....9
   3.1. Файлова структура (main branch)....9
   3.2. Сървърна част (API Endpoints)....9
   3.3. Клиентска част (Основни компоненти)....10
   3.4. База данни (Модели и Части)....12
   3.5. Тестване - тестове и покритие....13
4. Инфраструктура....14
   4.1. Инфраструктурна диаграма....14
   4.2. Docker конфигурация....14
   4.3. CI/CD pipeline....15
      4.3.1. CI файлове и алгоритъм....15
      4.3.2. CD файл и алгоритъм....16
   4.4. Инструкции за стартиране на проекта....16
   4.5. Observability....16
5. Екранни снимки....17
6. Използвани AI инструменти....19
7. Заключение....19
8. Източници....19

Описание на проекта

- Уеб-базиран калкулатор, който помага на потребителите да оценят своя екологичен отпечатък въз основа на ежедневните им навици.
- Потребителят въвежда информация за дейности от ежедневието си:
  - Метод на транспорт (кола, обществен транспорт, велосипед, ходене пеша и др.)
  - Колко често използва избрания транспорт
  - Консумация на вода (приблизително дневно или седмично)
  - Употреба на пластмаса (бутилки, опаковки и др.)
  - Други ежедневни навици, които влияят върху околната среда
- Въведената информация се запазва в профил, който може да бъде редактиран въз основа прогреса на даден потребител.
- Системата изчислява приблизителен коефициент на влияние върху околната среда на база въведената информация.
- Архитектурата ще е микросървисна, за момента основните ни микросървиси за login система, Proxy API, API към AI за анализ на входните данни.

1. Проучване

1.1. Предметна област и Целева аудитория

- Предметна област - Екология, анализ на влиянието на обикновен гражданин върху околната среда.
- Целева аудитория - Деца и пълнолетни, желаещи да помогнат на природата.

1.2. Преглед на съществуващи решения

- WWF (Сайт за измерване екологичен отпечатък)
  - Идеята му е подобна на нашата, потребителят въвежда входни данни за своите покупки, как се храни и с какво, транспорта, който използва, и други условия.
  - След въвеждането на данни сайтът изчислява отпечатъка в глобални хектари (gha) и показва колко "планети Земя" биха били необходими, ако всеки потребител живееше така.
- MyClimate (Сайт за измерване на въглероден отпечатък)
  - Отново се попълва такъв тип въпросник за нашето ежедневие, каква енергия използваме и как и т.н.
  - Изчислява въглеродните емисии в еквивалент на C02 (tC02e), използвайки научна информация за факторите им.

- Предлага на потребителя начини за компенсация чрез действие и участие в проекти.
- Основна разлика, Плюсове и Минуси
  - Освен в типа отпечатък и мерната единица, основната разлика между двете е в обхвата и целта.
    - WWF е по-широк (обхваща екосистеми и други природни ресурси) и целта му е осъзнаване на начина на живот
    - MyClimate е с по-тесен обхват, фокусът му е анализ на факторите, измерване на заплахата и вземане на мерки.
  - WWF
    - Плюсове - По-широк обхват, по-цялостен екологичен модел (не се ограничава до въглеродни емисии), визуализира проблема с идеята за "планетите Земя"
    - Минуси - Силно обобщен модел, не предлага директна компенсация
  - MyClimate
    - Плюсове - Основата му е изградена с научна информация и е по-обемиста и по-конкретна, анализира факторите за емисиите и дава конкретни действия за компенсация
    - Минуси - Ограничен до въглеродния отпечатък, по-труден е за разбиране, резултатите са приблизителни, не точни

1.3. Tech Stack

- Backend: Java със Spring Boot (Maven) - Лесен е за конфигурация и писане, познат синтаксис, ясна логика, имаме опит от миналото приложение (сайта за пещерите на карта)
- Frontend: Vanilla (чист JavaScript, HTML и CSS) - Не сме използвали задълбочено framework-ове за frontend, добро разбиране за писане на код
- База данни: PostgreSQL - Релационна, лесна за интеграция локално и при контейнеризиране, опит от началото на 11-ти клас
- Контейнеризация: Docker
- Оркестрация: Kubernetes (Helmchart, CI/CD pipeline с GitHub Actions)

2. Проектиране

2.1. Функционални изисквания към системата / програмата

- РС (Разработка на Софтуер)

- - Да разделя бизнес логиката от достъпа до данните и frontend-а
- - Да използва Dependency Injection за свързване на компонентите
- - Да позволява въвеждане и редакция на входни данни от потребителя
- - Да изчислява екологичния отпечатък
- - Да визуализира резултатите
- - Да дава препоръки за опазване на околната среда

- ООП (Обектно-Ориентирано Програмиране)

- - Да прилага Single Responsibility принципа (всеки клас да има една отговорност)
- - Да прилага Open/Closed принципа, лесно да се добавят / махат нови решения
- - Да прилага Dependency Inversion, да съдържа интерфейси вместо конкретни класове
- - Да няма важна информация в Java контролерите
- - Да използва custom exceptions и да има unit / integration тестове
- - Да синхронизира достъпа до споделени ресурси

- БД (Бази Данни)

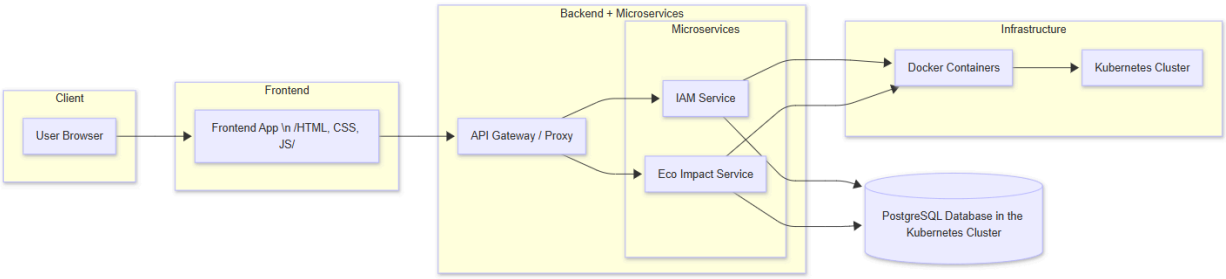
- - Да съхранява потребителите и техните входни данни
- - Да съхранява емисионните фактори (в отделна таблица с константи)
- - Да поддържа релационна база данни
- - Да поддържа миграции на базата
- - Да поддържа lazy / eager loading

- ВОТ (Виртуализация и Облачни Технологии)

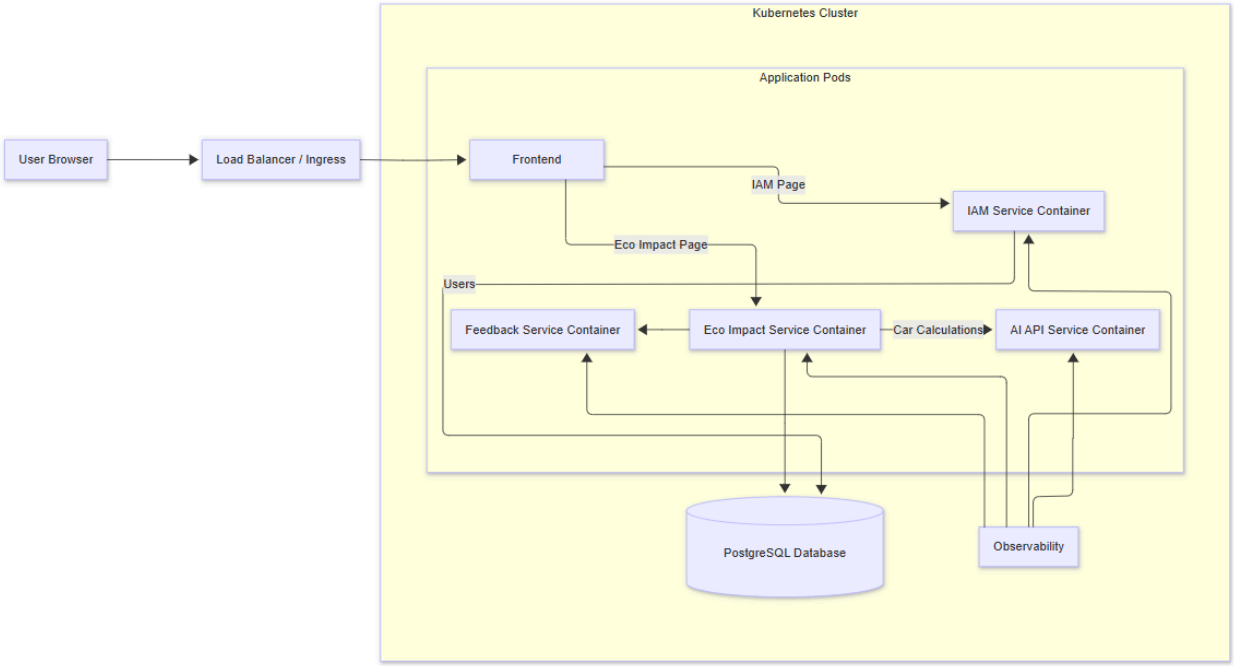
- - CI/CD pipeline с автоматично build-ване и тестване
- - Да изпраща известие при неуспешен build
- - Да бъде контейнеризирана чрез Docker
- - Да бъде оркестрирана в Kubernetes Cluster
- - Да поддържа Observability (Grafana - OpenTelemetry чрез Grafana Alloy)
- - Да събира логове

- - Да поддържа alerting
- - Да предоставя метрики
- - Да използва Infrastructure as Code (IaC)

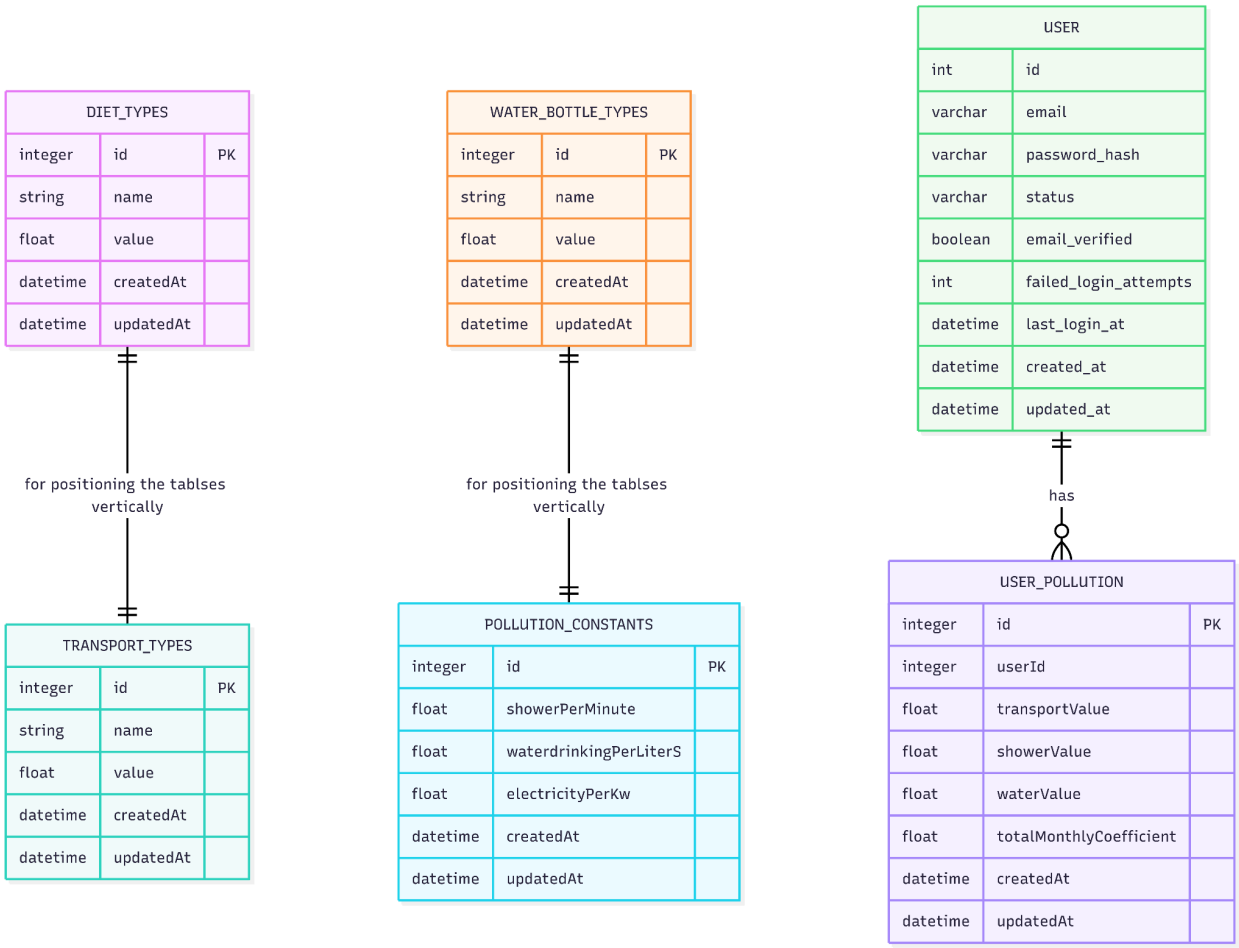
2.2. Архитектура на системата



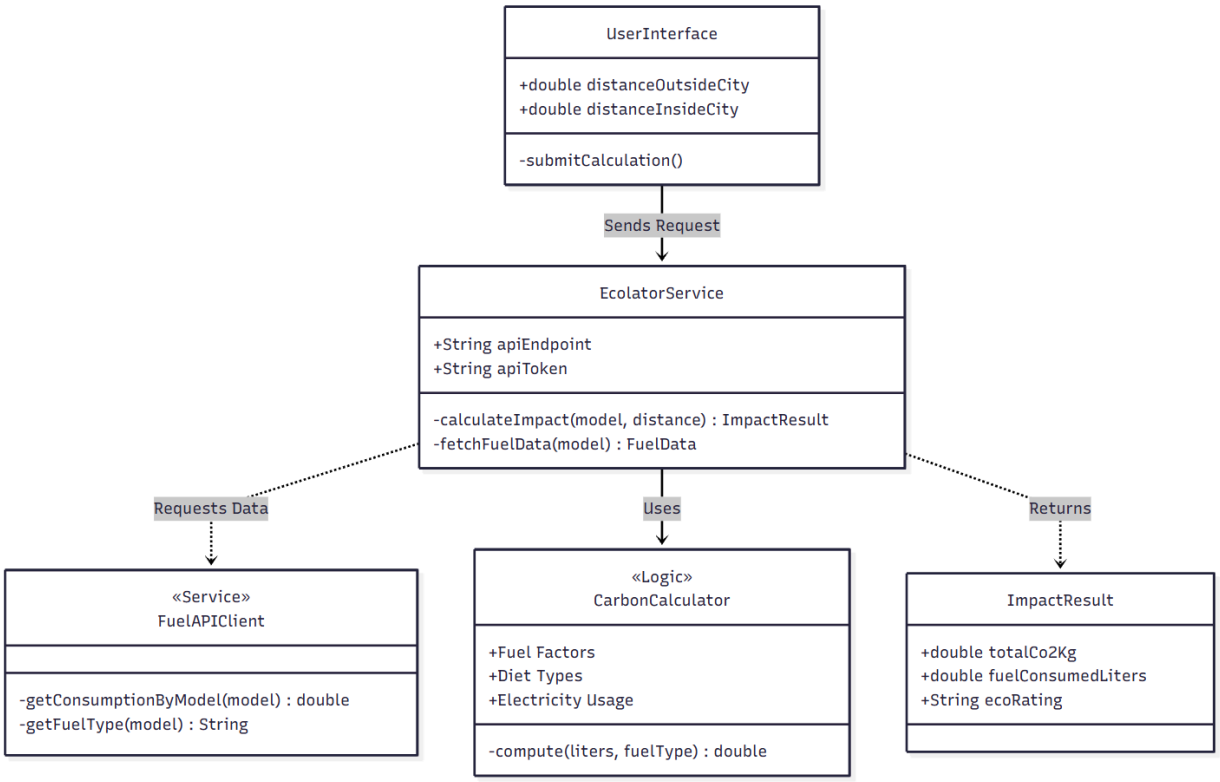
2.3. Инфраструктурна диаграма



2.4. Схема на БД



2.5. UML Class диаграма



Алгоритъм - Потребителят подава данни за начина си на хранене (всеяден, вегетарианец, веган) към service-а EcolatorService, който използва логиката на CarbonCalculator, изчислява отпечатъка и му бива върнат с ImpactResult под формата на въглеродните му емисии, влиянието му на околната среда и процента, в който се намира.

3. Реализация

3.1. Файлова структура (main branch)

```
Root-Square-Ecolator/
├── .github/
│   └── workflows/
├── .vscode/
├── car-fuel-api/
├── diagrams/
├── ecolator-chart/
├── ecolator/
├── static/
├── .gitignore
├── LICENSE
└── README.md
```

3.2. Сървърна част (API Endpoints)

Service	Порт	Отговорност
eco-impact (ecolator)	8080	Основна бизнес логика — изчисляване на емисии, управление на потребителски данни, REST API
ai-api (car-fuel-api)	8085	Интеграция с Groq AI за определяне на разхода на гориво по модел на автомобил
iam-service	8081	Автентикация и оторизация (NodePort 30081)
PostgreSQL	5432	Релационна база данни — потребителски данни, референтни данни

Grafana Alloy	4317/4318/12345	Observability collector — traces, metrics, logs към Grafana Cloud

3.3. Клиентска част (Основни компоненти):

IAM API:

Има IAM система под формата на отделно API направено на Java + Spring boot + Spring Security. Системата поддържа роли които не са използвани в текущия проект, както и JWT stateless сесии. За хеширане е използван BCrypt алгоритъм

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Registers a new user; returns JWT.
POST	/api/auth/login	Public	Validates credentials; returns JWT.
GET	/api/users/me	Authenticated	Returns current user profile and audit data.
POST	/api/users/change-password	Authenticated	Updates password if old password is valid.
GET	/api/admin/users	Admin Only	Retrieves list of all registered users.
PUT	/api/admin/users/{id}/unlock	Admin Only	Re-activates a locked account.

Това са някои от endpoint-ите за това API.

AI Fuel Consumption API:

Сървисът изпраща структуриран prompt към Groq AI (llama-3.3-70b-versatile) и парсва отговора. Промпт правилата изискват отговор само с числова стойност.
Следните инструкции се изпращат с всеки prompt:
What is the average combined fuel consumption (city + highway) of the %s in litres per 100 kilometres (L/100 km)?
Rules:

- Reply with ONLY the numeric value, nothing else (e.g. 7.2).
- Use the combined/mixed cycle figure.

- If the exact model year is unknown, use the most common/recent variant.
- If the car is electric or hydrogen, reply with 0.
- Never add units, words, or punctuation — just the bare number.

Алгоритъм за изчисляване на отпечатъка

Общото CO_2 = Транспорт CO_2 + Електричество CO_2 + Диета CO_2

Транспортни емисии:

```
litersUsed = (distanceKm * litersPer100Km) / 100
transportCO2 = litersUsed * fuelEmissionFactor
```

Емисионни фактори по вид гориво (kg CO_2 / литър):

Гориво	Фактор (kg CO_2/L)
Petrol (бензин)	2.31
Diesel (дизел)	2.68
LPG (газ)	1.51

Емисии според начина на хранене (kg CO_2/месец):

Диета	Емисии (kg/месец)
Omnivore	80.0
Vegetarian	45.0

Vegan	25.0

Емисии на електричество: electricityCO2 = kWh × 0.50 kg/kWh

Процент: percentile = round((totalCO2 / 300.0) × 100), ограничен между 1 и 99.

Системата генерира автоматични препоръки при:

- cityTransportKm > 100 — предложение за обществен транспорт
- outsideCityTransportKm > 150 — предложение за споделен транспорт
- Разход > 9 L/100km (град) или > 7 L/100km (извън) — предложение за по-ефективен автомобил
- Диета "omnivore" — предложение за намаляване на месото
- electricityKwh > 200 — предложение за енергоефективни уреди

3.4. База данни (Модели и Заявки):

Проектът използва ORM (Object-Relational Mapping) технология за управление на взаимодействието между Java обектите и базата данни. ORM-ът автоматично преобразува данни между релационния формат на базата данни и обектния формат на Java приложението.

Системата е организирана в следните слоеве:

Модели (Model Layer)

Java класове, които представляват таблиците в базата данни. Всеки модел съответства на една таблица и съдържа полетата и логиката на валидация за тази таблица.

Складове за данни (Repository Layer)

Интерфейси, които осигуряват готови операции за работа с данните (четене, писане, актуализирането, изтриване). Това елиминира необходимостта от писане на SQL заявки.

Transfer Objects (DTO Layer)

Класове, които служат за пренос на данни между фронтенда и бекенда. DTOs съдържат само необходимите полета за определен API endpoint, без излишни детайли.

При стартиране, ReferenceDataSeeder попълва референтните таблици (ако са празни) с предефинирани стойности:

- Диети: Vegan (2.5), Vegetarian (3.0), Pescatarian (3.5), Low Meat (4.5), Medium Meat (5.5), High Meat (7.0)
- Транспорт: Walking (0.0), Bike (0.01), Bus (0.08), Metro (0.05)
- Бутилки: Plastic 0.5L (0.08), Plastic 1L (0.15), Glass 0.5L (0.12), Reusable (0.02)
- Pollution Constants: shower=0.08/min, water=0.0003/L, electricity=0.40/kWh

3.5. Тестване - тестове и покритие:

Проектът съдържа unit и integration тестове и за двата основни сървиса.

eco-impact Тестове

Test Class	Покритие
EmissionServiceUnitTest	Unit тест на EmissionService с мокнат repository — проверява CO_2 изчислението, персентила, препоръките и съхранението в БД
EmissionControllerUnitTest	Unit тест на EmissionController — проверява делегирането към service
ReferenceDataSeederUnitTest	Проверява seed логиката — seed само при празна БД, пропуска при съществуващи данни
JpaRepositoryIntegrationTest	Integration тест с H2 in-memory БД — проверява всички repositories
EcolatorIntegrationTests	Full stack MockMvc тест — POST /api/emissions/calculate, проверява отговора и персистентността

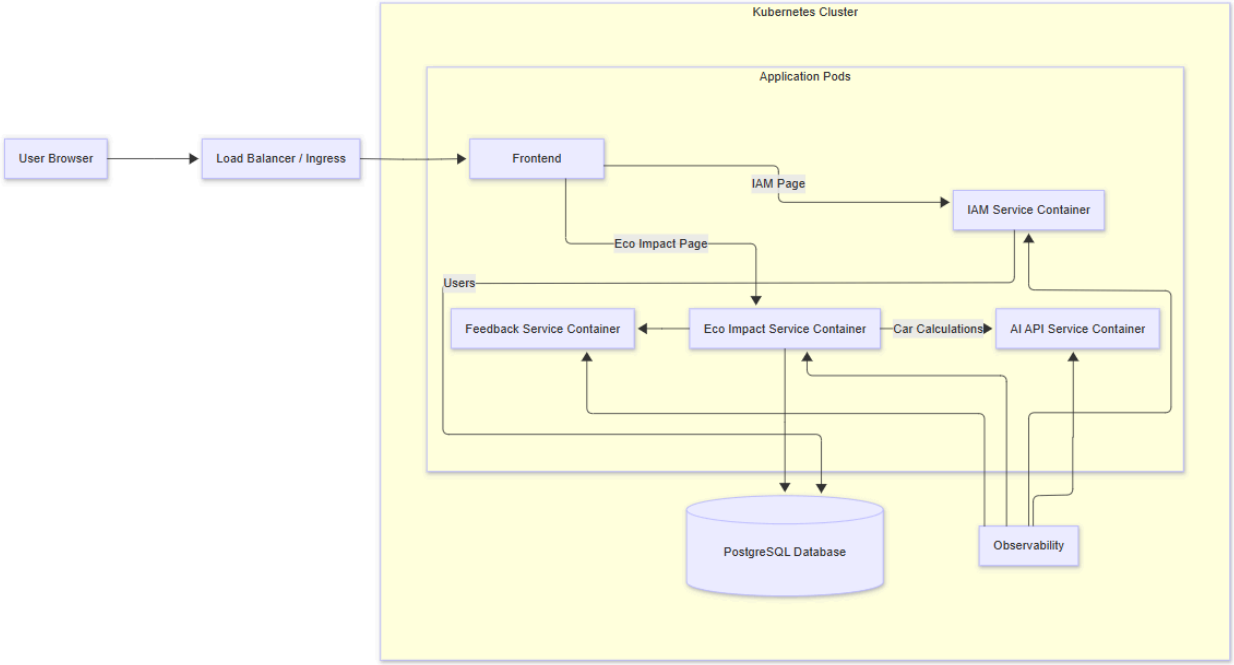
ai-api Тестове

Test Class	Покритие
FuelLookupServiceUnitTest	Unit тест на buildPrompt() и parseNumber() с ReflectionTestUtils
FuelLookupServiceIntegrationTest	Integration тест с MockWebServer — симулира Groq API responses, проверява request format и error handling
FuelControllerTest	Unit тест на FuelController — проверява валидация и делегирането към service
FuelApiIntegrationTest	Full stack WebTestClient тест — POST и GET endpoints, bad request cases

GroqRequestTest / GroqResponseTest	Unit тестове на model класовете

4. Инфраструктура

4.1. Инфраструктурна диаграма



4.2. Docker конфигурация

Deployments

Deployment	Replicas	Resources	Probes
eco-impact	2	200m/256Mi req, 500m/512Mi limit	/actuator/health/liveness + /readiness
ai-api	2	200m/256Mi req, 500m/512Mi limit	/actuator/health/liveness + /readiness

postgres	1	500m/512Mi req, 1/1Gi limit	pg_isready exec probe
iam	1	—	—
alloy (observability)	1	—	—

Services

Service	Тип	Порт(ове)
eco-impact	ClusterIP	8080
ai-api	ClusterIP	8085
postgres	ClusterIP	5432
iam	NodePort	8081 (NodePort: 30081)
alloy	ClusterIP	4317 (gRPC), 4318 (HTTP), 12345 (UI)

4.3. CI/CD Pipeline

4.3.1. CI файлове и алгоритъм

- .github/workflows/eco-ci.yaml → CI за главното приложение (eco-impact)
- .github/workflows/ai-api-ci.yaml → CI за AI API (ai-api)
  Алгоритъм: Когато има промяна в директориите ecolator или car-fuel-api, се задействат съответните потоци. Основното, което правят, е Build и Push на image-а viktorsirakov08/eco-impact:v1.0.x или viktorsirakov08/ai-api:v1.0.x към

Docker Hub с автоматична актуализация на номера на версията от GitHub Actions бот (github-actions в commit-ите) във values.yaml файла в ecolator-chart

4.3.2. CD файл и алгоритъм

- .github/workflows/cd.yaml → CD за Helm Chart-а в Kubernetes Cluster-а
  Алгоритъм: Когато има промяна във values.yaml файла в ecolator-chart (това включва от github-actions бота през CI потоците), се изчаква одобрението на self-hosted actions-runner, актуализира Helm Chart-а и го deploy-ва в Kubernetes Cluster-а.

4.4. Инструкции за стартиране на проекта

- git clone https://github.com/TUES-2026-PBL-11-klas/Root-Square-Ecolator.git
- cd Root-Square-Ecolator
- Пускане на Kubernetes Cluster-а
- kubectl create secret generic db-secret -n ecolator --from-literal=POSTGRES_DB=ecolator --from-literal=POSTGRES_USER=ecolator --from-literal=POSTGRES_PASSWORD=ecolator
- helm upgrade --install ecolator ./ecolator-chart -n ecolator --create-namespace -f ./ecolator-chart/values.yaml

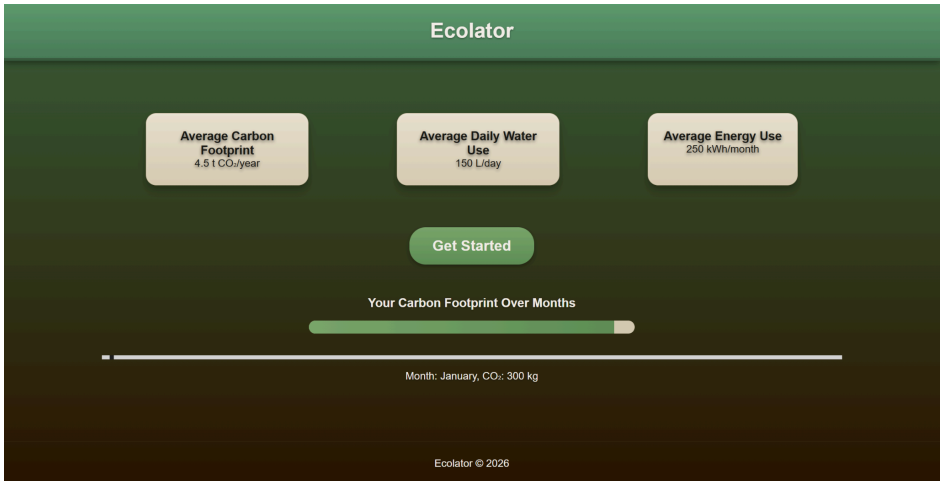
4.5. Observability

Линк към dashboard-а на програмата:
https://viktorsirakov08.grafana.net/goto/cfjla8j0uyy9sa?orgId=stacks-1600957

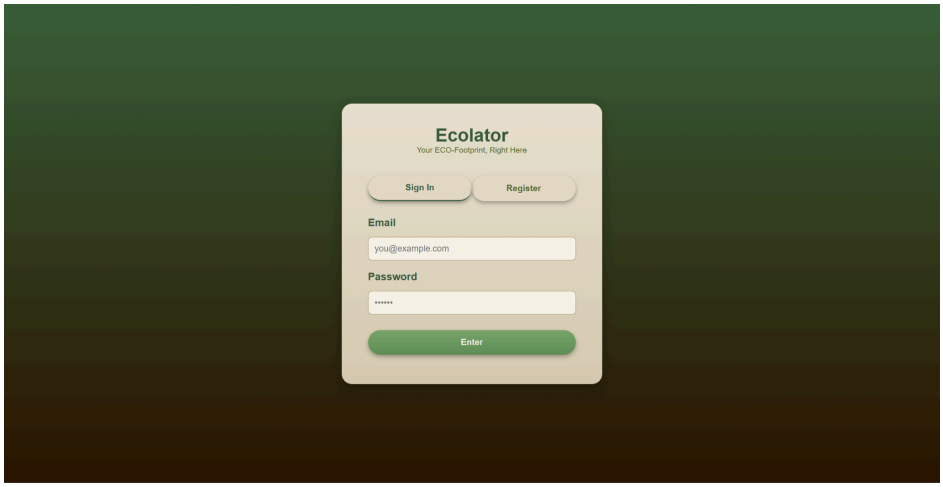
5. Екранни снимки

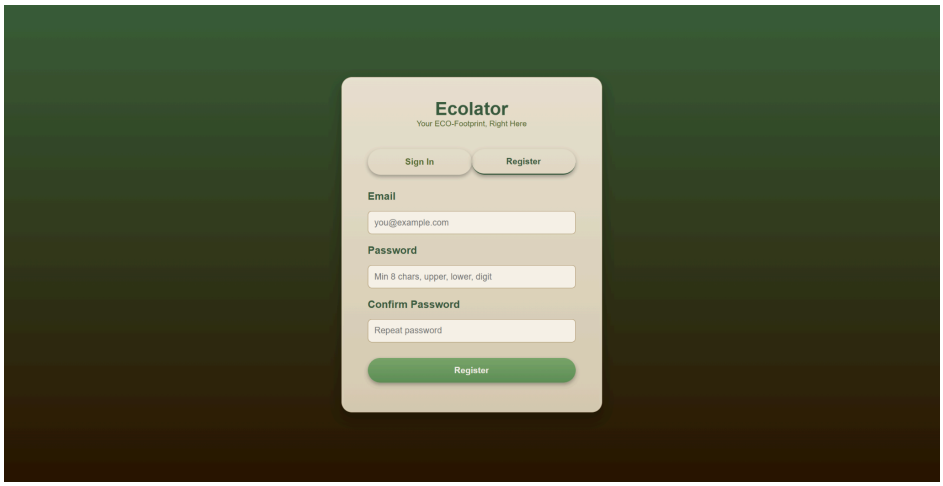
UI дизайн

Get Started:

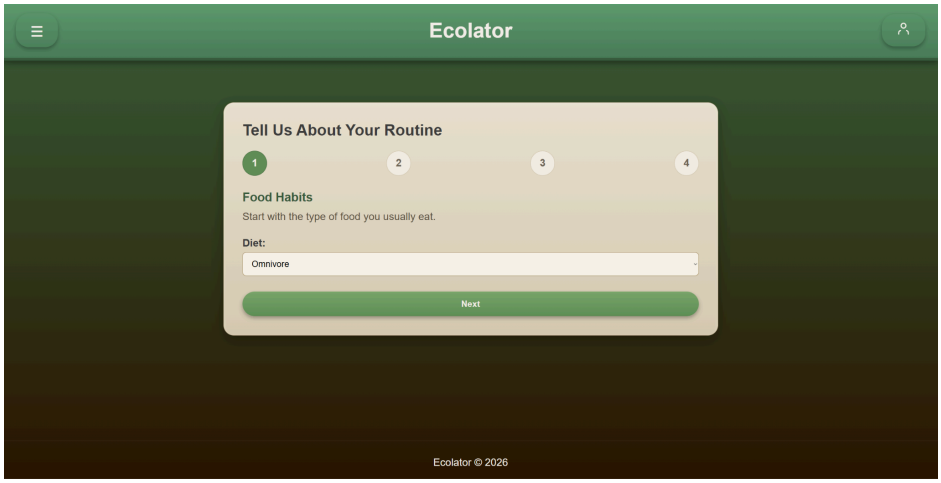


Login / Register:

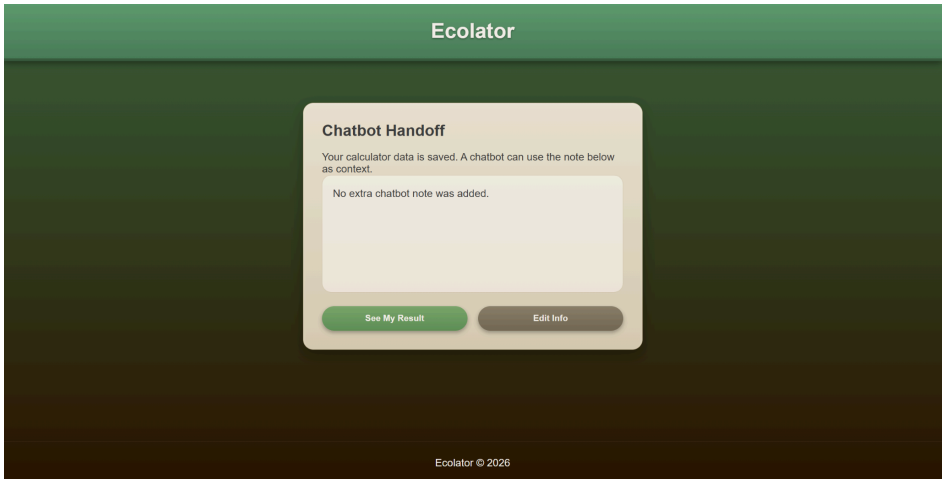




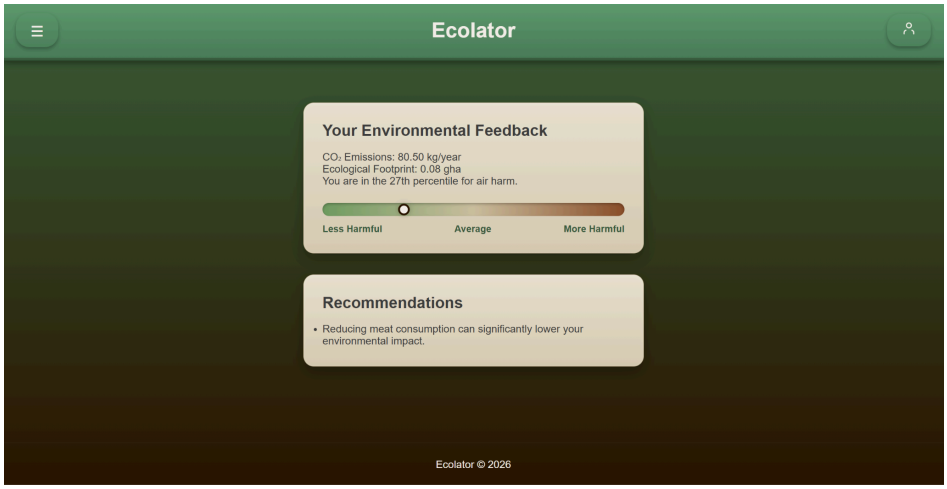
User Information:



Notes for the Chatbot:



Results & Feedback:



6. Използвани AI инструменти

- Claude AI, Gemini:
  - Boilerplate кодове за Dockerfile-овете
  - Unit & Integration тестовете
  - Дебъгване & Навързване

7. Заключение

- Развитие на умения с Docker и Kubernetes
- Добиване на опит в правенето на Unit & Integration тестове
- За първи път правим Observability
- Успешно интегриран IAM Service
- Успешно интегриран Helm Chart
- Добър старт; Силен завършек

8. Източници

- https://www.worldwildlife.org → WWF (Подобно съществуващо решение)
- https://www.wwf.bg → WWF България
- https://www.myclimate.org → MyClimate (Подобно съществуващо решение)
- https://grafana.com → Observability